



Resolución de problemas en Mathematica

I. Definición de un vector:

- Para definir un vector en Mathematica utilizamos la sintaxis: $a = \{componente\ x, componente\ y\}$

Ejemplo:

```
a = {2,1}; (*Define un vector con componente en x = 2 y componente y = 1*)  
b = {x,y}; (*Define un vector generalizado con componentes x, y*)
```

Listing 1: Definiendo vectores

- Para evaluar un vector generalizado en un punto específico, podemos utilizar la notación especial aprendida en la primera práctica

Ejemplo:

```
b/.{x->1, y->1}  
{1,1}
```

Listing 2: Evaluando un vector generalizado

II. Operaciones vectoriales:

Al trabajar con vectores podemos sumarlos, restarlos y escalarlos multiplicandolos por un escalar, estas serían las operaciones básicas utilizando los operadores aritméticos correspondientes.

- Adición y resta de vectores $C = A \pm B$

```
a + b  
{2+x, 1+y}  
  
(*Si quisieramos evaluar b en un punto*)  
a - b/.{x->1, y->1}  
{1,0}
```

Listing 3: Operaciones básicas

- Escalar un vector αA

```
5*a  
{10,5}
```

Listing 4: Operaciones básicas

III. Vector unitario:

Los vectores unitarios son importantes para expresar las componentes de un vector en dirección de estos, tienen la cualidad de que su norma es igual a 1.

- **Normalize[vector]** permite calcular el vector unitario del vector que recibe. Es decir, devuelve un vector.
- Por otra parte, el comando **Norm[vector]** permite calcular la norma o magnitud del vector que recibe. Es decir, devuelve un número.

Ejemplo:

```
Normalize[a] (*Calcula el vector unitario de a*)
{2/Sqrt[5], 1/Sqrt[5]}

Norm[a] (*Calcula la norma de a*)
Sqrt[5]
```

Listing 5: Vector unitario y norma de un vector

IV. Integral de campo eléctrico

Encontrar el campo eléctrico correspondiente a una línea infinita de carga ubicada en el eje z.

1. Primero definimos la posición del punto de campo y la posición de la distribución de carga:

```
$Assumptions = {\[Rho] >= 0, \[Rho]prim >= 0}; (*Le especificamos a
  mathematica que \[Rho] es una cantidad positiva*)

(*Posicion del punto de campo*)
r[\[Rho]_, \[Phi]_, z_] = {\[Rho], 0, 0};

(*Posicion de la distribucion de carga*)
r'[\[Rho]prim_, \[Phi]prim_, zprim_] = {0, 0, zprim};
```

2. Armamos el argumento de la integral de la forma: $E = \frac{1}{4\pi\epsilon_0} \int_{-\infty}^{\infty} \frac{r - r'}{R^3} \lambda dL'$

```
CampoLinea = \[Lambda]/(4*Pi*\[Epsilon])*
  Integrate[(r[\[Rho], \[Phi], z] - r'[\[Rho]prim, \[Phi]prim, zprim])/
    Norm[r[\[Rho], \[Phi], z] - r'[\[Rho]prim, \[Phi]prim, zprim]]^3, {zprim,
      -Infinity, Infinity}] // Simplify

{\[Lambda]/(2 \[Pi] \[Rho] \[Epsilon]), 0, 0}
```

Finalmente encontramos el campo eléctrica para una línea infinita en el eje z: $E = \frac{\lambda}{2\pi\epsilon_0\rho}\hat{\rho}$

V. Producto escalar

Una de las operaciones básicas que podemos realizar con vectores es el producto escalar o producto punto, este se interpreta como la medida de cuanto contribuye un vector en dirección de otro.

- El comando **Dot[vector 1, vector2]** recibe 2 vectores y devuelve el producto punto entre ellos.

Ejemplo: Sabemos que $F = qE$, calcule el trabajo $dW = F \cdot ds$, realizado por el campo eléctrico de un plano infinito $E = \frac{\sigma}{2\epsilon_0} \frac{z}{|z|} \hat{z}$ sobre una carga q que se desplaza en dirección del eje $+\hat{z}$.

```
CampoE = {0, 0, (\[Sigma]*z)/(2*Abs[z]*\[Epsilon])};
F = q*CampoE;
dS = {0, 0, 1}; (Desplazamiento en direccion de z unitario)

dW = Dot[F, dS]; (*Diferencial de trabajo*)
W = Integrate[dW, {z, 0, 1}] (*Trabajo realizado por E*)
(q*Sigma)/(2*epsilon)
```

Listing 6: Producto punto

- Adicionalmente podemos utilizar la notación alternativa **A . B** la cual mathematica interpreta como producto punto.

Ejemplo:

```
a . b (*Utilizando los vectores anteriormente definidos*)
2x + y
```

Listing 7: Notación alternativa del producto punto

VI. Producto vectorial

El producto vectorial o producto cruz, es una operación entre dos vectores que genera un tercer vector el cual es perpendicular al plano formado por los dos vectores que operamos.

- El comando **Cross[vector1, vector2]** recibe dos vectores y devuelve un tercer vector perpendicular a ambos. **ADVERTENCIA:** Los vectores deben tener 3 componentes espaciales debido a que el producto cruz se define como una operación en 3 dimensiones.

Ejemplo: El torque eléctrico es una medida de la torsión que siente un dipolo en presencia de un campo eléctrico. Se define como $\tau = p \times E$ donde $p = qd$ con d siendo la distancia entre las cargas que conforman un dipolo. Calcule τ utilizando el campo eléctrico ya definido para un plano infinito y un dipolo apuntando en dirección del $\hat{x}$

```
p = {q*d,0,0}; (*Dipolo en direccion x unitario*)
tau = Cross[p,CampoE] (*CampoE definido anteriormente*)
{0, -(q*d*z*Sigma)/(2*epsilon*|z|), 0}
```

Listing 8: Producto cruz

VII. Operadores vectoriales

Los operadores vectoriales son aquellos que nos permiten determinar información sobre el comportamiento de un vector o campo vectorial cuando los aplicamos sobre estos. Mediante el operador `nabla` se definen operaciones como el gradiente, divergencia y el rotacional.

- El gradiente **Grad[F[var1, var2, var3], {var1, var2, var3}]** recibe una función escalar y devuelve un vector que apunta en la dirección donde f crece más rápido, el gradiente es perpendicular a las curvas de nivel de f .

Ejemplo:

```
f[x_,y_] := x^2 + y^2;  
Grad[f[x,y], {x,y}]  
{2 x, 2 y}
```

Listing 9: Gradiente de un vector

- La divergencia **Div[vector, {var1, var2, var3}]** recibe un con 3 componentes espaciales y devuelve la densidad de flujo que atraviesa determinada región o punto de un campo.

Ejemplo:

```
r = {3*x^7 + y*z, x^14 y^2, z + y + x};  
Div[r, {x, y, z}]  
1 + 21 x^6 + 28 x y
```

Listing 10: Divergencia de un vector

- El rotacional **Curl[vector, {var1, var2, var3}]** recibe un vector con 3 componentes espaciales y devuelve un vector que mide la tendencia que sufre ese vector a girar o circular alrededor de determinada región o punto.

Ejemplo:

```
F = {3*x*y^2 + z, x^8*y*z, z};  
{-x^8 y, 1, -6 x y + 8 x^7 y z}
```

Listing 11: Rotacional de un vector

- El laplaciano **Laplacian[F[var1, var2, var3], {var1, var2, var3}]** mide el nivel de curvatura de un campo con relación a las variaciones medias alrededor de la región.

Ejemplo:

```
Laplacian[F, {x, y, z}]  
{6 x, 56 x^6 y z, 0}
```

Listing 12: Laplaciano

VIII. Gráficos vectoriales en 2D

- **VectorPlot**[vector,{var1, mín, máx}, {var2, mín, máx}] permite visualizar campos vectoriales en 2D.

Ejemplo:

```
VectorPlot[b, {x, -1, 1}, {y, -1, 1}]
```

Listing 13: VectorPlot

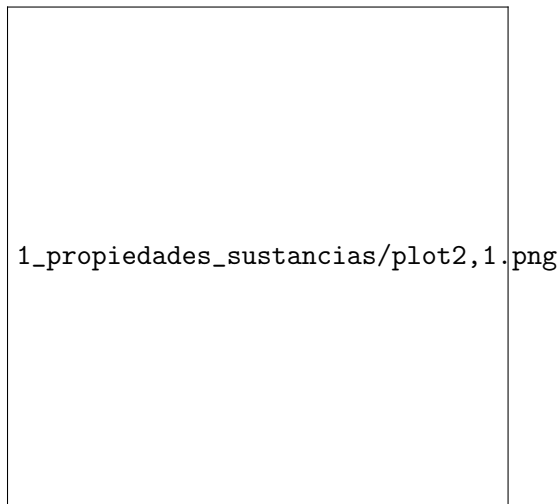


Figura 1: Visualización usando VectorPlot[]

- **StreamPlot**[vector,{var1, mín, máx}, {var2, mín, máx}] permite visualizar un campo vectorial con sus respectivas líneas de flujo o curvas tangentes al campo vectorial.

Ejemplo:

```
StreamPlot[b, {x, -1, 1}, {y, -1, 1}]
```

Listing 14: StreamPlot

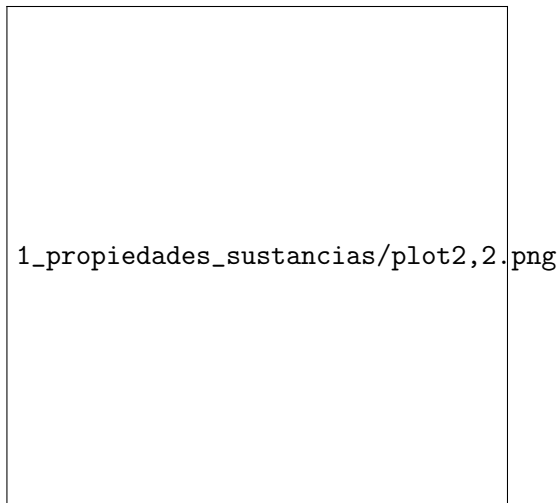


Figura 2: Visualización usando StreamPlot[]

- **ContourPlot**[funcion[var1,var2], {var1, mín, máx}, {var2, mín, máx}], nos permite graficar las curvas de nivel de una función escalar de 2 variables, recordar que las curvas de nivel de una función de la forma $z = f(x, y)$ serán los valores que puede tomar z . El comando **Contours** → **cantidad** nos permite controlar el número de curvas que deseamos visualizar en el gráfico de contornos.

Ejemplo:

```
ContourPlot[f[x,y], {x,-1,1},{y,-1,1}, Contours->50]
```

Listing 15: Gráfico de contornos

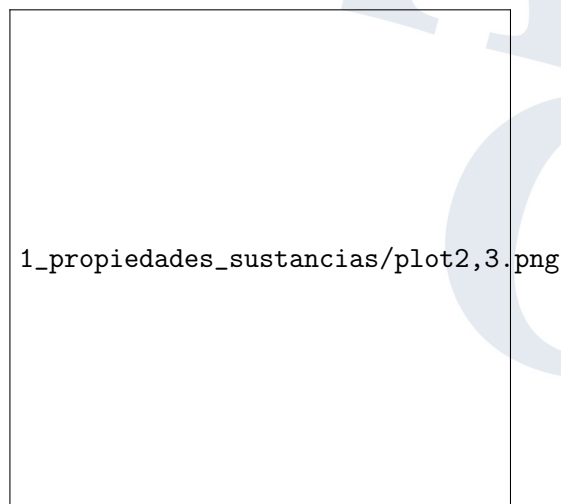


Figura 3: Visualización curvas de nivel

IX. Gráficos vectoriales en 3D

- `Plot3D[funcion[var1, var2], {var1, mín, máx}, {var2, mín, máx}]` permite graficar una función en el espacio tridimensional, con el añadido de que Mathematica nos da la libertad de ir rotando el gráfico para visualizarlo de mejor manera.

Ejemplo:

```
Plot3D[x^2 + y^2, {x, -1, 1}, {y, -1, 1}]
```

Listing 16: Gráfico 3D

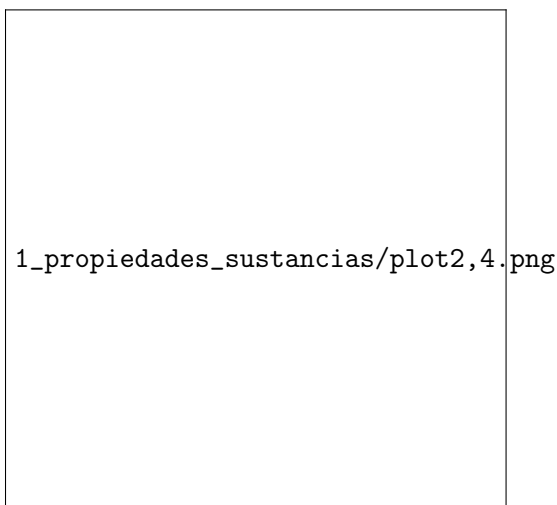


Figura 4: Curva en el espacio tridimensional

- De igual manera que podemos graficar en 3D, también podemos crear gráficos dinámicos con el comando `Manipulate[Plot[funcion,{var1,min,max},{var2,min,max}], {par1,min,max}, {par2,min,max}]`. Al igual que con el `Plot3D`, también podemos rotar este gráfico.

Ejemplo:

```
Manipulate[Plot3D[a*x^2 + b*y^2, {x, -1, 1}, {y, -1, 1}], {a, -1, 1}, {b, -1, 1}]
```

Listing 17: Manipulate en 3D

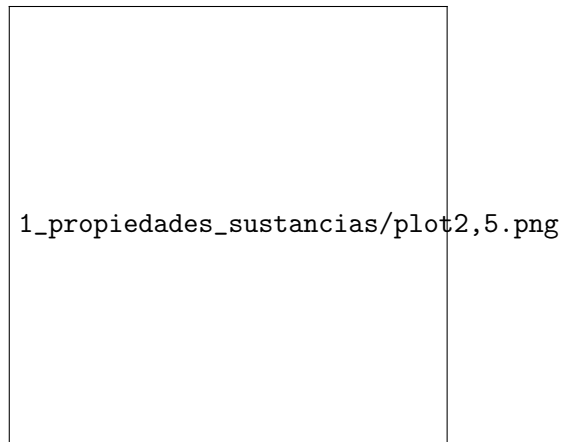


Figura 5: Manipulando un gráfico en 3D

- **VectorPlot3D**[vector,{var1,mín,máx},{var2,mín,máx},{var3,mín,máx}] permite visualizar un campo vectorial en 3D dimensiones espaciales, al igual que todo gráfico en 3D, lo podemos rotar para mayor comodidad visual.

Ejemplo:

```
R = {x,y,z}
VectorPlot3D[R,{x,-1,1},{y,-1,1},{z,-1,1}]
```

Listing 18: VectorPlot3D

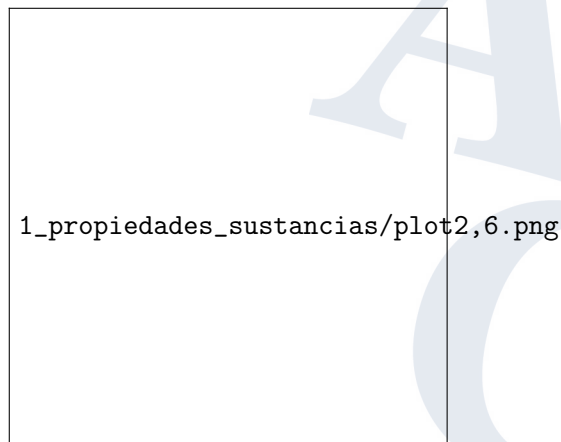


Figura 6: Campo vectorial en 3D

- **StreamPlot3D**[vector,{var1,mín,máx},{var2,mín,máx},{var3,mín,máx}] Permite graficar el campo vectorial en 3D y ver las líneas de flujo en el espacio tridimensional.

Ejemplo:

```
StreamPlot3D[R,{x,-1,1},{y,-1,1},{z,-1,1}]
```

Listing 19: StreamPlot3D

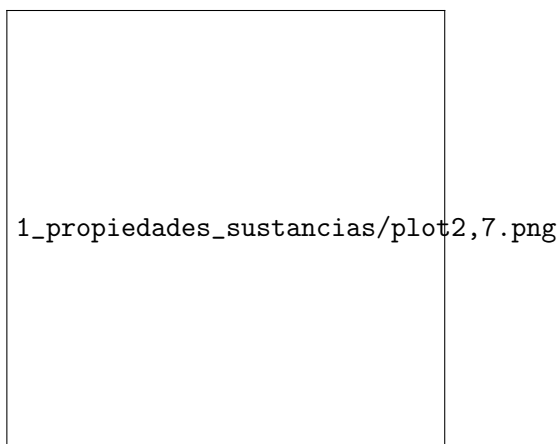


Figura 7: Líneas de flujo en 3D

- **ContourPlot3D**[funcion == nivel, {var1,mín,máx},{var2,mín,máx}, {var3,mín,máx}]
permite visualizar superficies de nivel, a diferencia del comando utilizado anteriormente donde solo podíamos ver las curvas de nivel.

Ejemplo:

```
ContourPlot3D[{x^2+y^2+z^2==1, x^2+y^2+z^2==2, x^2+y^2+z^2==3}, {x,-2,2}, {y,-2,2}, {z,-2,2}]
```

Listing 20: ContourPlot3D

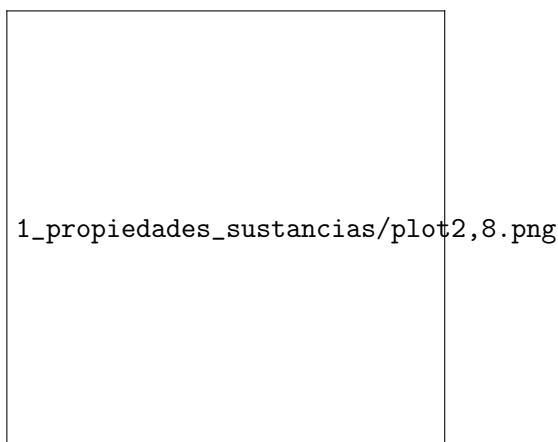


Figura 8: Superficies de nivel

X. Cambio de sistemas coordenados

Otra herramienta útil de la cual disponer al momento de trabajar con campos vectoriales, son las transformaciones de un sistema coordenado a otro para simplificar la complejidad de los problemas. Los 3 sistemas coordenados más utilizados son las coordenadas cartesianas, coordenadas polares, coordenadas cilíndricas y coordenadas esféricas. Mathematica por defecto trabaja en coordenadas cartesianas pero si lo requerimos podemos pasar de un sistema a otro.

- El comando `TransformedField["Sistema de origen" → "Sistema destino", Campo Vectorial, {var1,var2,var3} → {var4,var5,var6}]` nos permite pasarnos de un sistema coordenado a otro pero tenemos que tener mucho cuidado en que nuestras variables destino estén completamente limpias sin valores residuales guardados porque sino Mathematica no efectuara de forma correcta la instrucción. Para pasar de un sistema coordenado a otro utilizamos las funciones ya nombradas por mathematica:

1. *Cartesian*
2. *Cylindrical*
3. *Spherical*

Ejemplos:

```
TransformedField["Cartesian" -> "Cylindrical", x^2 + y^2 + z^2, {x, y, z} ->
  {\[Rho],\[Phi],Z}] // Simplify

Z^2 + \[Rho]^2 (*Nuevo campo vectorial en coordenadas cilindricas*)
```

Listing 21: Coordenadas Cartesianas a Coordenadas Cilíndricas

```
TransformedField["Cartesian" -> "Spherical", x^2 + y^2 + z^2, {x, y, z} ->
  {r, \[Theta], \[Phi]}} // Simplify

r^2 (*Nuevo campo vectorial en coordenadas esfericas*)
```

Listing 22: Coordenadas Cartesianas a Coordenadas Esféricas

```
Clear[{x, y, z}] (*Primero limpiamos las variables porque las usamos
  anteriormente*)

TransformedField["Cylindrical" -> "Cartesian", {\[Rho]*Cos[[Phi]], \[Rho]*Sin
  [[Phi]],0}, {\[Rho],\[Phi],Z} -> {x, y, z}] // Simplify

{(x^2 - y^2)/Sqrt[x^2 + y^2], (2 x y)/Sqrt[x^2 + y^2], 0}
(*Nuevo campo vectorial en coordenadas cartesianas*)
```

Listing 23: Coordenadas Cilíndricas a Coordenadas Cartesianas

XI. Ley de Gauss

Utilizando la ley de Gauss encontraremos el campo eléctrico para una línea infinita que reposa en el eje z . Lo haremos paso por paso, sabiendo de antemano que E solo tendrá componente en la dirección radial debido a que en la dirección z se extiende hasta el infinito y en dirección ϕ es indivisible del valor que se escoja. Utilizamos la ecuación $\int_s E \cdot da = \frac{Q_{ennc}}{\epsilon_0}$ donde $dQ = \lambda dL$.

1. Primero definimos el campo eléctrico tomando en cuenta lo anterior mencionado:

```
Campoelctrico = {componenteRadial, 0, 0};
```

2. Procedemos a calcular el flujo a través de cada cara superficial de nuestra superficie Gaussiana. La superficie que mejor se acopla es la de un cilindro por lo que trabajaremos en coordenadas cilíndricas:

```
(*Flujo en la cara z=-1*)
z0 = Integrate[\[Rho]*(Campoelectrico . {0, 0, -1}),{\[Rho],0,\[Rho]},{\[Phi],
0, 2Pi}] (*recordar da = \[Rho]d\[Rho]d\[Phi](-z)*)
0

(*Flujo en la cara z=1*)
z1 = Integrate[\[Rho]*(Campoelectrico . {0, 0, 1}),{\[Rho],0,\[Rho]},{\[Phi]
],0,2Pi}] (*recordar da = \[Rho]d\[Rho]d\[Phi](z)*)
0

\[Rho]1 = Integrate[\[Rho]*(Campoelectrico . {1, 0, 0}),{\[Phi],0,2Pi},{z,-L/2,L
/2}] (*recordar da = \[Rho]d[Phi]dz(\[Rho])*)
2 componenteRadial L \[Pi] \[Rho]
```

3. Procedemos a sumar la contribución de cada cara y a resolver para nuestra componenteRadial del campo E que estamos buscando. Como podemos observar es el resultado deseado.

```
Solve[z0 + z1 + \[Rho]1 == (\[Lambda] L \[Epsilon]), componenteRadial] //
Simplify (*Recordar dQ = \[Lambda] dL *)

{{componenteRadial -> \[Lambda]/(2 \[Pi] \[Rho] \[Epsilon])}}
```

XII. EXTRA

Si desearamos ver gráficamente nuestra respuesta podemos utilizar comandos para gráficos en 3D como aprendimos:

1. Primero transformemos el campo que encontramos en coordenadas cilíndricas a coordenadas cartesianas:

```
Clear[x, y, z] (*Limpiamos las variables destino*)
TransformedField["Cylindrical" -> "Cartesian",{\[Lambda]/(2*Pi*\[Epsilon]*\[
Rho]), 0, 0}, {\[Rho], \[Phi], Z} -> {x, y, z}]

{(x \[Lambda])/(2 \[Pi] (x^2 + y^2)\[Epsilon]), (y \[Lambda])/(2 \[Pi] (x^2 +
y^2)\[Epsilon]), 0}
```

2. Gráficamos la superficie de Gauss y el campo eléctrico:

```
superficieGauss = RegionPlot3D[x^2 + y^2 <= 1, {x,-1,1}, {y,-1,1}, {z,-1,1},
PlotStyle -> {Blue, Opacity[0.15]}];

campovectorial = VectorPlot3D [{x/(2*Pi*(x^2 + y^2)), y/(2*Pi*(x^2 + y^2)),
0}, {x,-1,1},{y,-1,1}, {z,-1,1}]; (*Hicimos \[Lambda]=1 y \[Epsilon]=1*)

Show[superficieGauss, campovectorial]
```



Figura 9: Dirección del campo eléctrico